

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

# TAREFA 3: INFLUÊNCIA DA ESTRUTURA DE LAÇOS E OTIMIZAÇÃO DE COMPILADORES NO NÍVEL DE INSTRUÇÃO

*Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 14/08/2026.*

## 1. EXPERIMENTO

*"Implemente três laços em C para investigar os efeitos do paralelismo ao nível de instrução (ILP):*

- *1) inicialize um vetor com um cálculo simples;*
- *2) some seus elementos de forma acumulativa, criando dependência entre as iterações;*
- *3) quebre essa dependência utilizando múltiplas variáveis.*

*Compare o tempo de execução das versões compiladas com diferentes níveis de otimização (00, 02, 03) e analise como o estilo do código e as dependências influenciam o desempenho."*

## 2. CÓDIGO

O código abaixo tem como objetivo principal avaliar o impacto do Paralelismo em Nível de Instrução (ILP) através da medição precisa do tempo de execução de diferentes abordagens em laços de repetição.

Inicialmente, o programa define a função auxiliar "recuperar_tempo_instantaneo", que utiliza a chamada de sistema "clock_gettime" configurada com o relógio monotônico do Linux. Essa escolha garante marcações

de tempo em segundos com alta precisão e imunes a alterações no relógio do sistema operacional, sendo ideal para medir o tempo de processamento.

Na função principal, o programa realiza a alocação dinâmica (via malloc) de um vetor de 5 milhões de elementos do tipo double. Esse tamanho específico de vetor foi escolhido para garantir que o tempo de execução das operações atinja, no mínimo, a escala de milissegundos, conforme orientado em sala de aula. Essa escala é necessária para obter medições de tempo consistentes, evitando que flutuações microscópicas do sistema operacional mascarem os resultados do experimento.

A primeira etapa do experimento consiste na inicialização do vetor. Através de um laço simples, o programa itera sobre todas as posições, preenchendo cada espaço com o valor de seu respectivo índice multiplicado por 0.5. Essa fase serve como uma linha de base computacional para a escrita sequencial em memória.

A segunda etapa implementa a soma acumulativa tradicional. Neste laço, cada elemento do vetor é somado a uma única variável (soma_dependente). Essa estrutura cria uma forte dependência de dados entre as iterações: o processador é obrigado a aguardar o resultado da iteração atual para poder iniciar o cálculo da próxima. Essa dependência inerente ao algoritmo torna-se um gargalo, pois impede a CPU de utilizar seus múltiplos módulos de execução para paralelizar as instruções.

Para contornar essa limitação e evidenciar o benefício do ILP, a terceira etapa quebra a dependência de dados utilizando a técnica de desenrolamento de laço (loop unrolling). O laço é modificado para avançar de quatro em quatro posições a cada iteração, distribuindo as somas em quatro acumuladores independentes (soma_independente1 a soma_independente4). Como o cálculo de um acumulador não depende do valor do outro, a arquitetura superescalar do processador consegue despachar e executar essas quatro operações de adição simultaneamente. Ao final do laço, as quatro sub-somas são combinadas para gerar o resultado final.

**Figura 1: Código utilizado para testes**

```c
#define _POSIX_C_SOURCE 199309L // Necessário para clock_gettime() no Linux
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Tamanho do vetor: 5 milhões de elementos double (aprox. 40 MB)
#define N 5000000
```

```c
// Recuperar tempo de relógio transcorrido (em segundos) conforme orientado na sala de aula
double recuperar_tempo_instantaneo() {
    struct timespec ts; // Estrutura para armazenar o tempo
    clock_gettime(CLOCK_MONOTONIC, &ts); // Obtém o tempo de relógio monotônico (não retrocede)
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9; // Converte para segundos
}

int main() {
    // Alocação dinâmica do vetor para evitar estouro de pilha (stack overflow)
    double *vetor = (double *)malloc(N * sizeof(double));
    if (vetor == NULL) {
        fprintf(stderr, "Erro: Falha na alocação de memória.\n");
        return 1;
    }

    double inicio, fim;
    double tempo_inicializacao, tempo_soma_dependente, tempo_soma_independente;

    // 1) Inicialização do vetor com um cálculo simples ---
    inicio = recuperar_tempo_instantaneo();
    for (size_t i = 0; i < N; i++) {
        vetor[i] = (double)i * 0.5;
    }
    fim = recuperar_tempo_instantaneo();
    tempo_inicializacao = fim - inicio;

    // 2) Soma acumulativa com dependência forte entre iterações ---
    double soma_dependente = 0.0;
    inicio = recuperar_tempo_instantaneo();
    for (size_t i = 0; i < N; i++) {
        soma_dependente += vetor[i];
    }
    fim = recuperar_tempo_instantaneo();
    tempo_soma_dependente = fim - inicio;

    // 3) Soma acumulativa quebrando a dependência com múltiplas variáveis ---
    // Desenrolado por um fator de 4 utilizando 4 acumuladores independentes (ILP)
    double soma_independente1 = 0.0;
    double soma_independente2 = 0.0;
    double soma_independente3 = 0.0;
    double soma_independente4 = 0.0;

    inicio = recuperar_tempo_instantaneo();
    for (size_t i = 0; i < N; i += 4) {
        soma_independente1 += vetor[i];
        if (i + 1 < N) soma_independente2 += vetor[i + 1];
```

```
        if (i + 2 < N) soma_independente3 += vetor[i + 2];
        if (i + 3 < N) soma_independente4 += vetor[i + 3];
    }
    fim = recuperar_tempo_instantaneo();
    tempo_soma_independente = fim - inicio;

    double soma_independente = soma_independente1 + soma_independente2 + soma_independente3
+ soma_independente4;

    // Exibição dos tempos e dos resultados (evita que o compilador otimize/remova os laços)
    printf("Tamanho do vetor: %d elementos\n", N);
    printf("1) Tempo de Inicializacao:              %f segundos\n", tempo_inicializacao);
    printf("2) Tempo da Soma Acumulativa (dependente): %f segundos (Soma: %.2f)\n",
tempo_soma_dependente, soma_dependente);
    printf("3) Tempo da Soma Multi-var (independente): %f segundos (Soma: %.2f)\n",
tempo_soma_independente, soma_independente);

    // Libera a memória alocada
    free(vetor);
    return 0;
}
```

## 3. COMPILAÇÃO

Para compilar e executar o experimento, utilizou-se o compilador GCC com o comando base gcc -O[nível] -Wall -Wextra -std=c11 main.c -lm -o main && ./main.

O foco da análise, no entanto, está na alteração do nível de otimização do compilador. Utilizou-se -O0 como linha de base (sem otimizações), mantendo a tradução literal do código para evidenciar o gargalo das dependências de dados. Em seguida, -O2 para otimizações moderadas, que melhoram a performance usando registradores e reorganizando instruções de forma segura. Por fim, -O3 para otimizações agressivas, ativando recursos avançados como loop unrolling automático e vetorização.

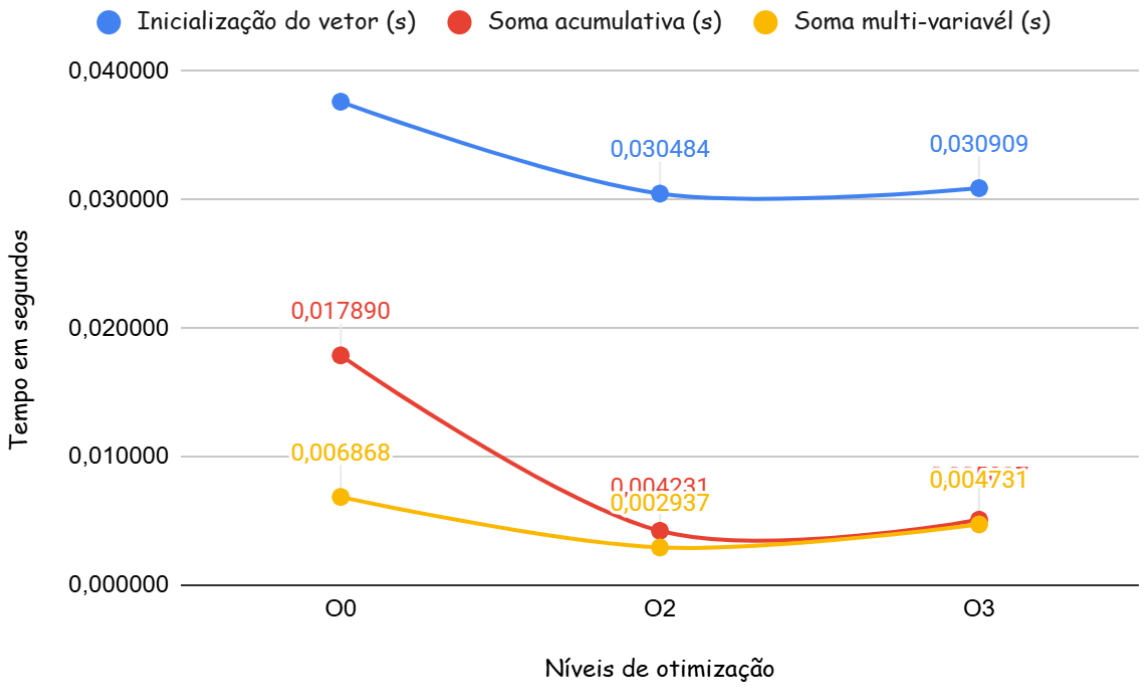
## 4. RESULTADOS

O resultado dos testes do programa gerado pelo código da Figura 1, podem ser vistos na tabela (Tabela 1) e gráfico abaixo (Figura 2).

**Tabela 1: Dados do experimento**

| Nível de otimização | Inicialização do vetor (s) | Soma acumulativa (s) | Soma multi-variável (s) |
|---|---|---|---|
| O0 | 0,037637 | 0,017890 | 0,006868 |
| O2 | 0,030484 | 0,004231 | 0,002937 |
| O3 | 0,030909 | 0,005097 | 0,004731 |

**Figura 2: Evolução do tempo das operações com a mudança de níveis**




Os resultados evidenciam uma clara diferença na natureza computacional das etapas. A inicialização do vetor consumiu o maior tempo absoluto (cerca de 0,030s), pois a escrita sequencial de 40 MB de dados é uma tarefa limitada pela velocidade da memória RAM, onde a CPU tem pouca margem de atuação. Já as operações de soma registraram tempos muito menores (de 0,017s a 0,002s), pois são estritamente limitadas pelo processador, permitindo que a estrutura do código e as escolhas de compilação mostrem seu verdadeiro impacto.

Avaliando o papel do compilador, o nível “-O0” comprova o valor prático do ILP onde quebrar a dependência de dados reduziu o tempo de 0,017 s para 0,006 s. O nível “-O2” obteve o melhor desempenho do experimento (0,0029 s na soma

multi-variável) ao aliar a reescrita inteligente do código com o uso eficiente de registradores.

Entretanto, o “nível -O3” registrou tempos piores que o “-O2”. Esse fenômeno, conhecido na arquitetura de computadores, ocorre porque o “-O3” aplica heurísticas muito agressivas. Ao tentar vetorizar o código forçadamente (SIMD) ou aplicar um desenrolamento de laço (*loop unrolling*) excessivo, o compilador pode ter gerado um excesso de instruções. Isso frequentemente causa dois problemas: o executável fica grande demais e "estoura" o cache de instruções ultrarrápido do processador (*I-cache miss*), ou esgota a quantidade de registradores físicos disponíveis na CPU, forçando o armazenamento temporário de variáveis na memória RAM (*register spilling*). Em ambos os casos, a sobrecarga gerada por essas "superotimizações" acabou prejudicando a execução fluida do laço.

## 5. CONSIDERAÇÕES FINAIS

O experimento demonstrou de forma prática a importância do Paralelismo em Nível de Instrução (ILP) e o impacto direto das dependências de dados no desempenho do software. Ficou evidente que a intervenção do programador ao estruturar o código (como a quebra manual de dependências utilizando múltiplas variáveis) é uma técnica altamente eficaz para explorar a arquitetura superescalar dos processadores, resultando em ganhos expressivos de velocidade mesmo sem a ajuda de otimizações automáticas.

Além disso, a análise dos compiladores revelou que delegar a otimização exclusivamente à ferramenta exige cautela. Embora o nível “-O2” tenha entregue o melhor equilíbrio e os menores tempos de execução gerais, a regressão de desempenho observada no nível “-O3” serviu como um alerta, indicando que otimizações agressivas e automáticas não são uma solução mágica e podem gerar efeitos colaterais indesejados, como a sobrecarga do cache de instruções ou o esgotamento de registradores.